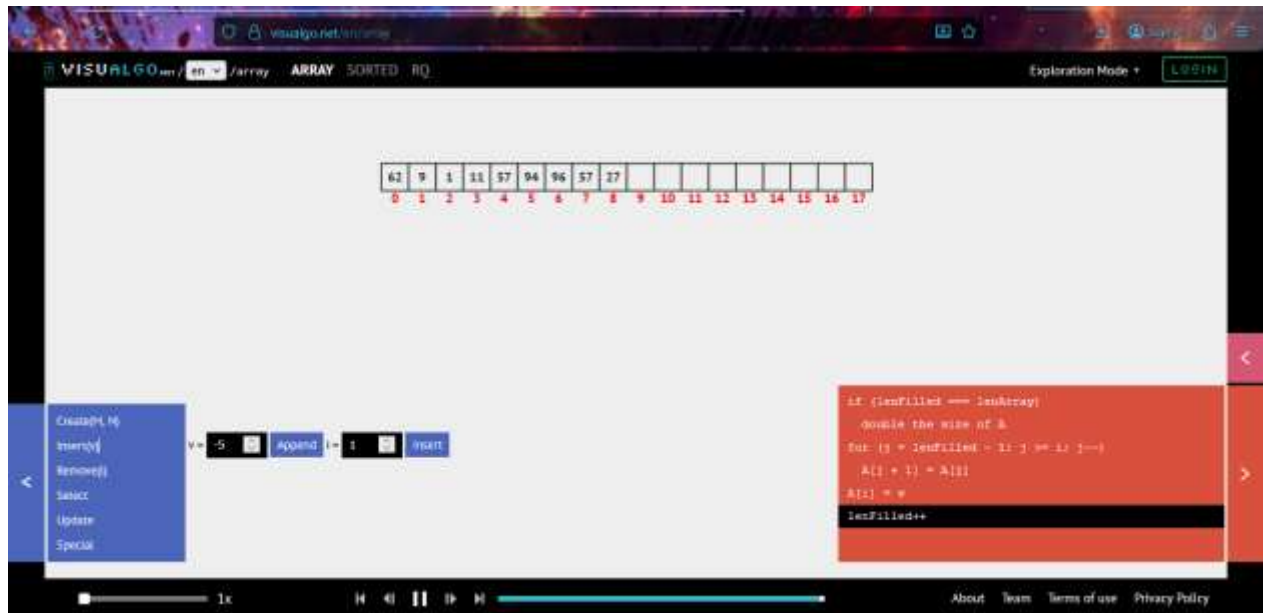


МАСИВИ – ПРЕГОВОР С ДОПЪЛНЕНИЕ

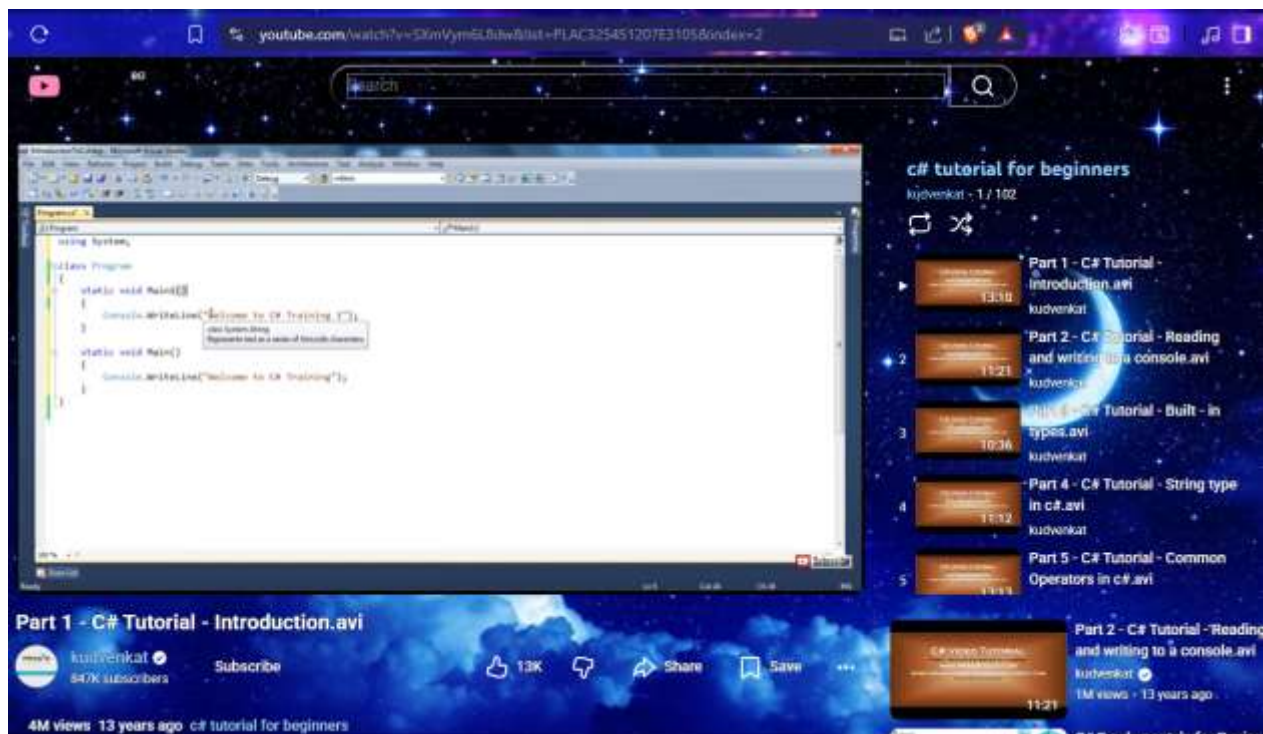
Масив `Array[]` е структура от данни = **последователност от елементи от един и същ тип**, подредени в паметта и достъпни чрез индекс. **Индексите започват от 0.**

За масиви – визуализация на алгоритмите: <https://visualgo.net/en/array>



Спряганият за най-добър туториъл за учене на C#:

<https://www.youtube.com/watch?v=SXmVym6L8dw&list=PLAC325451207E3105&index=1>



```
// обикновено имената са в мн.ч. numbers, fruits, cars...
1v int[] numbers1v = new int[5]; // нов масив вариант 1: 1v

2v int[] numbers2v; // деклариране 3 варианта за създаване
   numbers2v = new int[6]; // 6 елемента, по подразбиране са 0-и

3v int[] numbers3v = { 7, 14, 21, 28, 35, 42};
   // ПРОМЯНА на ел.
   Console.WriteLine(numbers3v[0]); // първи елемент 7
   numbers3v[2] = 10; // промяна на трети елемент 21 -> 10
                       // не пази старата стойност

for (int i = 0; i < numbers1v.Length; i++) // обхожда елементите
{
    numbers1v[i] = int.Parse(Console.ReadLine()); // прочита и напълва
    // елементът = прочетеното
}

1) for (int i = 0; i < numbers3v.Length; i++) // обхожда елементите
{
    Console.WriteLine(numbers3v[i] + " "); // отпечатва
}
   // 2 вида обхождане - с for или foreach

2) foreach (int number in numbers3v) // друго обхождане
{
    Console.WriteLine(number + " "); // обикновено е името на масива в ед.ч.
}
}
```

Масивите са удобни, АКО предварително знаем дължината. Те са бързи, с малко вградени функции, можем да променяме елемент.

```
// обикновено имената са в мн.ч. numbers, fruits, cars...
int[] numbers1v = new int[5]; // нов масив вариант 1: 1v

int[] numbers2v; // деклариране
numbers2v = new int[6]; // 6 елемента, по подразбиране са 0-и

int[] numbers3v = { 7, 14, 21, 28, 35, 42};

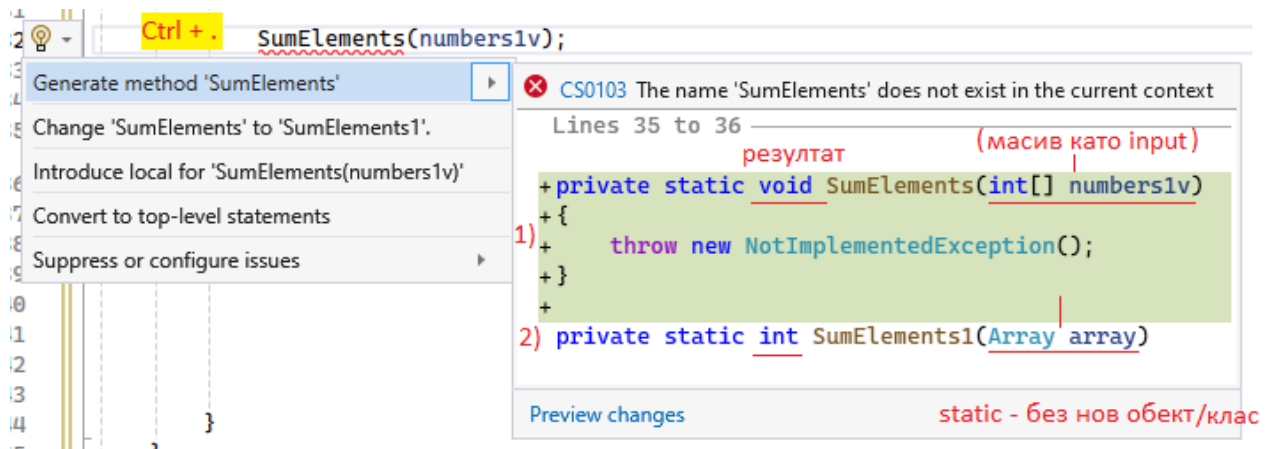
Console.WriteLine(numbers3v[0]); // първи елемент 7
numbers3v[2] = 10; // промяна на трети елемент 21 -> 10

for (int i = 0; i < numbers1v.Length; i++) // обхожда елементите
{
    numbers1v[i] = int.Parse(Console.ReadLine()); // прочита и натрупва
}
For (int i = 0; i < numbers3v.Length; i++) // обхожда елементите
{
    Console.Write(numbers3v[i] + " "); // отпечатва
}

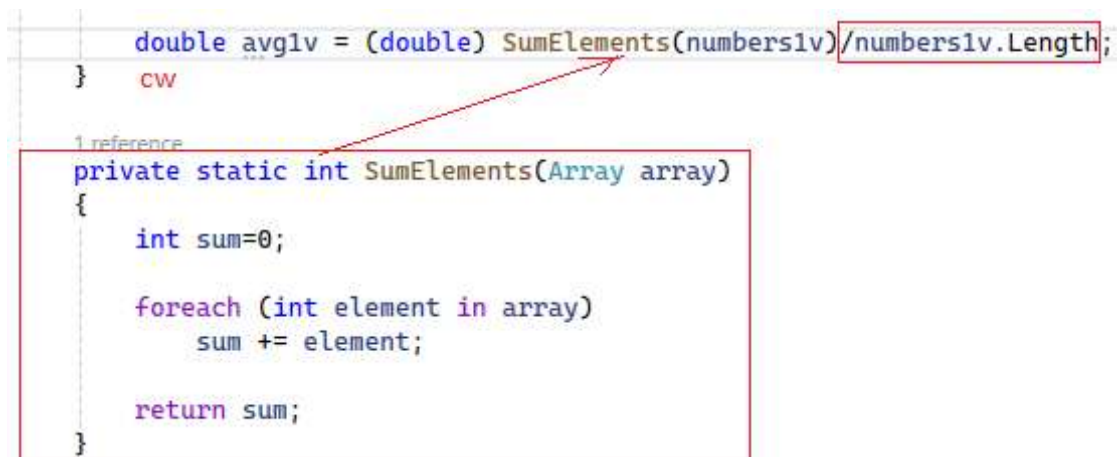
foreach (int number in numbers3v) // друго обхождане
{
    Console.Write(number + " "); // обикновено е името на масива в ед.ч.
}

}
```

Нека съберем елементите, но действието да бъде отделено в метод. Изписваме метода, подаваме му за обработка като input масив, след което си спестяваме писане с автокорекция **Ctrl + .** Предлага ни 2 начина, които можем да разгледаме – нека редактираме до втория, защото искаме сумирането да връща число `int`. Обърнете внимание, че се поставя на правилното място извън `Main()`:



Нека методът за сумиране бъде използван за изчисляване на средноаритметично по един по-демонстративен начин:



```
double avg1v = (double) SumElements(numbers1v)/numbers1v.Length;
Console.WriteLine(avg1v);
}

private static int SumElements(Array array)
{
    int sum=0;

    foreach (int element in array)
        sum += element;

    return sum;
}
```

Нека подадем 3-я масив, който е пълен със стойности и чрез метод изпишем неговите минимална и максимална стойности:



```
double avg1v = (double) SumElements(numbers1v)/numbers1v.Length;
Console.WriteLine(avg1v);

MinMaxValues(numbers3v);
}
```

Отново го постави на правилното място извън Main()

```
1 reference
private static void MinMaxValues(int[] numbers3v)
{
    throw new NotImplementedException();
}
```

```
1 reference формула
private static int SumElements(Array array)
{
    int sum=0;
    foreach (int element in array)
        sum += element;

    return sum;
}
```

Нека редактираме метода:

```
MinMaxValues(numbers3v);
}
```

подава стойност за изчисление

```
1 reference
private static void MinMaxValues(int[] arr)
{
    int min = arr[0];
    int max = arr[0];
    for (int i = 1; i < arr.Length; i++)
    {
        if (arr[i] < min) min = arr[i];
        if (arr[i] > max) max = arr[i];
    }

    Console.WriteLine($"Минимум: {min}, максимум: {max}");
}
```

нарочно различно
означение arr във
формулата

ако срещне по-малко

Ако е необходима промяна на броя или подредбата, най-добре е да се създаде втори нов масив, който да съхранява различieto.

Често използвани вградени методи:


```
Array.Sort(arr); // Сортиране
Array.Reverse(arr); // Обръщане
Array.IndexOf(arr, 4); // Търсене на индекс
Array.Resize(ref arr, 10); // Промяна на размер

arr.Length; // Дължина
arr.Rank; // Брой измерения
```

Остава да си подредим задачата и да я поукрасим – може да изпечатаме чрез **ASCII art**

<https://www.asciiart.eu/art-and-design/patterns>

```
Console.WriteLine(new string('#', 33)); // Променят оригинала!!!
Console.WriteLine("Променен елемент: " + string.Join(" ", numbers3v));
Array.Sort(numbers3v); // сортира
Console.WriteLine("След сортиране: " + string.Join(" ", numbers3v));
Array.Reverse(numbers3v); // преобръща
Console.WriteLine("След обръщане: " + string.Join(" ", numbers3v));
```

Използваме при изпечатване **string.Join()**, т.к. масивът е от числа **int**, а **конзолата работи само с текст** – тя трябва да отпечата текст и заедно с добавянето на интервал я преобразуваме.

```
Console.WriteLine(new string('#', 33));
Console.WriteLine("Променен елемент: " + string.Join(" ", numbers3v));
Array.Sort(numbers3v); // сортира
Console.WriteLine("След сортиране: " + string.Join(" ", numbers3v));
Array.Reverse(numbers3v); // преобръща
Console.WriteLine("След обръщане: " + string.Join(" ", numbers3v));

Console.WriteLine("Оригинал: " + string.Join(" ", numbers3v));
```

Също, обърнете внимание, че **Array.Sort()** и **Array.Reverse()** **променят оригиналната редица!!!** По същият начин **Array.Clear()** я изпразва (изтрива стойностите) наведнъж.

```
Microsoft Visual Studio Debug Console
7
12
76
34
23
19
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Редица 3:
7 14 10 28 35
#####
Редица 3:
7 14 10 28 35 42
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Средна стойност: 32.8
Минимум: 7, максимум: 42
#####
Променен елемент: 7 14 10 28 35 42
След сортиране: 7 10 14 28 35 42
След обръщане: 42 35 28 14 10 7
Оригинал: 42 35 28 14 10 7
Array.Sort() и Array.Reverse()
променят оригинала!!!

D:\Exercises\11 class\Lists\bin\Debug\net8.0\Lists.exe (process 26032) exited with code 0.
Press any key to close this window . . .
```

Как да прочетем цяла редица с числа без добавяне на всеки елемент поотделно с for?

Със златната комбинация `.Split().Select(int.Parse).ToArray()`.

```
Console.WriteLine("Въведете редица: ");
int[] numbers4v = Console.ReadLine().Split().Select(int.Parse).ToArray();
Console.WriteLine(numbers4v); // System.Int32[] конзолата работи само с текст, не числа!!!
Console.WriteLine(string.Join(' ', numbers4v)); затоа преобразуваме със string.Join()
Array.ForEach(numbers4v, x => Console.Write(x)); най-съкратено изпечатване
                                         може да се вкара и интервал или логика с ||, &&
```

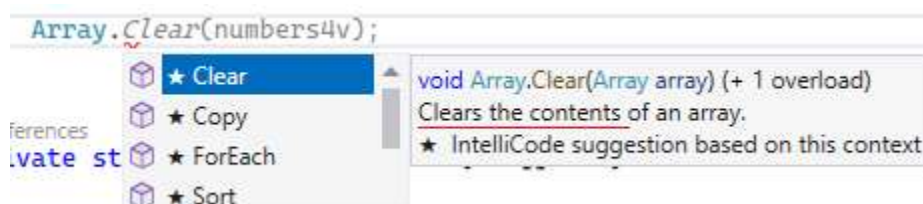
```
int[] numbers4v = Console.ReadLine().Split().Select(int.Parse).ToArray();

Console.WriteLine(numbers4v); // System.Int32[]
Console.WriteLine(string.Join(' ', numbers4v));

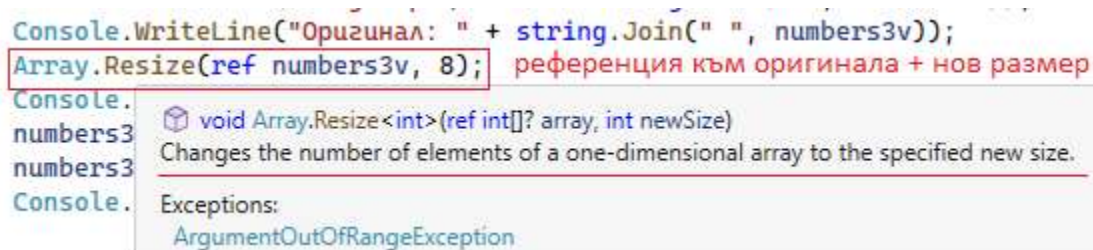
Array.ForEach(numbers4v, x => Console.Write(x));
```

```
Select Microsoft Visual Studio Debug Console
Въведете редица: 1 5 9
System.Int32[]
1 5 9
159
D:\Exercises\11 class\Lists\bin\Debug\net8.0
Press any key to close this window . . .
```

Ако разгледаме вградените методи за масиви Array, – те са малко:



Тестване на .Resize



Array.Resize() е статичен метод, наличен от .NET 2.0, който всъщност създава **нов масив** и копира елементите. За динамични колекции **List<T>** почти винаги е по-добрият избор поради по-оптималното управление на разширяването.

- Масивите са фиксирани, по-бързи за достъп и по-ефективни с паметта
- Списъците са динамични, предлагат богат API и са по-гъвкави
- Списъците използват масиви вътрешно, но добавят overhead за управление на капацитета
- Изборът зависи от конкретния случай - фиксиран размер и производителност VS динамичност и удобство

В повечето модерни приложения списъците са предпочитаният избор поради тяхната гъвкавост, а разликата в производителността често е незначителна за типични сценарии.

От кога в масивите има вграден метод `.Resize()` - в него се уточнява името на масива и новата дължина `Array.Resize(ref arr, arr.Length + 2);`



Отличен въпрос! Имам грешка в предния отговор - масивите нямат вграден метод `.Resize()` като инстанционен метод. Благодаря, че го посочихте!



```
int[] numbers = { 1, 2, 3 };

// Това НЕ работи:
// numbers.Resize(5); // ❌ Грешка - няма такъв метод

// Това работи:
Array.Resize(ref numbers, 5); // ✅ Правилно
Console.WriteLine(string.Join(", ", numbers)); // 1, 2, 3, 0, 0
```

допълва с 0 по подразбиране

Задкулисно създава нов масив с по-голям размер и копира в него.

```
// Това, което се случва зад кулисите:
int[] original = { 1, 2, 3 };
Array.Resize(ref original, 5);

// Еквивалентно на:
int[] temp = new int[5];           // 1. Създава нов масив
Array.Copy(original, temp, 3);     // 2. Копира старите елементи
original = temp;                   // 3. Пренасочва референцията
```

Използвами сме temp (от temporary) като временна променлива, която създаваме само, за да разменим 2 стойности, например

```
int a = 3, b = 5;  
temp = a; // temp = 3  
a = b;    // a = 5 – първото число е готово  
b = temp; // b = 3 – второто число е готово
```

Или когато копираме 1 файл, копието не може да стане със същото име като оригинала, затова временно трябва да го преименуваме (temp) и чак когато името е освободено, за да не се повтаря, можем да го зададем на новия файл (копието).

.Resize копира стария масив в по-голямо пространство, докато List просто добавя пространство, без да копира, с което спестява време и ресурси. .Resize създава нов масив, докато при списък се работи със стария (увеличен).

```
// Array.Resize създава НОВ масив и копира елементите  
// Това е операция O(n) и може да е бавна за големи масиви  
  
int[] largeArray = new int[1000000];  
  
// Лошо за производителността:  
for (int i = 0; i < 100; i++)  
{  
    Array.Resize(ref largeArray, largeArray.Length + 1); // ❌ Многократно копиране  
}  
  
// По-добре:  
List<int> list = new List<int>(1000000); // ✅ List управлява разширяването оптимално
```

Масивът е подходящ, ако предварително знаем размера, но не можем лесно да добавяме елемент в средата или края, докато при списъци можем.

```
// Предварително задаване на капацитет  
int[] array = new int[knownSize];
```

Списъкът е динамичен и с това по-гъвкав при разширяване и свиване:

```
// List<T> е специално проектиран за динамично разширяване  
List<int> list = new List<int>();  
list.Add(1); // Автоматично разширяване с оптимална стратегия
```

* .NET 1.0 (2002): Масиви са фиксирани, няма Resize

* .NET 2.0 (2005): Въведен Array.Resize() като помощен метод

Език	Подход	Преимущества	Недостатки
C#	<code>Array.Resize(ref arr, newSize)</code>	Удобен, безопасен	Създава нов масив
C	<code>realloc()</code>	Ефикасен, може да преизползва памет	Ръчно управление на паметта
C++	<code>std::vector.resize()</code>	Безопасен, автоматично управление	Overhead на контейнера
C++	ръчно с <code>new / delete</code>	Пълен контрол	Сложен, податлив на грешки

`realloc()` в C:

```
c
//  Предимства
- Може да преизползва същата памет ако има място
- Ефикасно за големи масиви
- Стандартна функция

//  Недостатъци
- Ръчно управление на паметта
- Податлив на memory leaks
- Новите елементи са неинициализирани
```

`std::vector` в C++:

```
c++
//  Предимства
- Автоматично управление на паметта
- Безопасен
- Богат API
- RAII (автоматично освобождаване)

//  Недостатъци
- Overhead (капацитет, size и др.)
- По-сложна вътрешна структура
```

- C: Използва `realloc()` за "resize", но изисква ръчно управление на паметта
- C++: Има `std::vector::resize()` което е безопасно и автоматично
- C#: Има `Array.Resize()` който е удобен но винаги създава нов масив

Препоръка: В C++ винаги предпочитайте `std::vector` над ръчно управление на масиви, освен в много специфични случаи, където е необходим пълен контрол над паметта.

Нека се върнем на списъци:

Списъци (Lists)

Дефиниция: Динамична колекция, която може да расте и свива.

Как създаваме списък:

```
List<int> numbers = new List<int>();
List<string> names = new List<string>() {"John", "Jane"};
```

Вградени функции:

```
List<int> list = new List<int>();

// Добавяне и премахване
list.Add(1);           // Добавяне в края
list.Insert(0, 5);     // Вмъкване на позиция
list.Remove(1);        // Премахване по стойност
list.RemoveAt(0);      // Премахване по индекс
list.Clear();          // Изчистване наведнъж

// Търсене и достъп
bool contains = list.Contains(5);
int index = list.IndexOf(5);
int count = list.Count; брой (дължина .Length)
int element = list[0];

// Други полезни методи
list.Sort();           търсачка, сочи към
list.Reverse();        всички x > 3
list.Find(x => x > 3); // Търсене с предикат
```

Бързина и Производителност

Операция	Масиви	Списъци
Достъп по индекс	⚡ $O(1)$ - най-бързо	⚡ $O(1)$ - почти също бързо
Добавяне в края	⊘ Неподдържано	⚡ $O(1)$ амортизирано
Вмъкване в средата	⊘ Неподдържано	🐢 $O(n)$ - бавно
Търсене	🐢 $O(n)$	🐢 $O(n)$
Сортиране	⚡ Бързо (на място)	⚡ Бързо (на място)

Списъците са базирани върху масиви:

```
// Ето как примерно изглежда List<T> вътрешно
public class List<T>
{
    private T[] _items;        // Вътрешен масив
    private int _size;         // Брой елементи
    private int _version;      // Версия за промени

    public List()
    {
        _items = Array.Empty<T>();
    }

    public void Add(T item)
    {
        if (_size == _items.Length)
        {
            EnsureCapacity(_size + 1); // Разширяване при необходимост
        }
        _items[_size++] = item;
    }

    private void EnsureCapacity(int min)
    {
        // Удвояване на капацитета или добавяне на минимум
        int newCapacity = _items.Length == 0 ? 4 : _items.Length * 2;
        Array.Resize(ref _items, newCapacity);
    }
}
```

Аспект	Масиви	Списъци
Основна памет	Точно колкото елементи	Още капацитет (обикновено 2x)
Overhead	Минимален	Допълнителни полета (_size, _version)
Фрагментация	Ниска	Променлива при разширяване

При увеличаване на броя на елементите, **масивът** се копира и добавя **точно толкова нови**, колкото е необходимо. При списъка, масивът, който стои в сърцевината му, също се копира на ново място,

НО при това дължината в паметта се увеличава (**първоначално двойно**), затова следващи увеличавания на размера се случват по-бързо и лесно, докато има свободна резервна памет. Също, списъците имат **допълнителна проверка за дължина и тип, което забавя**.

Т.е. виждаме, че списъците задкулистно представляват масиви.

Масивите в няколко измерения образуват многомерни **матрици**.

Масиви [n,m] или n x m се наричат матрици, в които имаме **[n - редове, m - колони]**.

```
// 3x3 матрица
int[,] matrix = new int[3, 3];
// 2x4 матрица
string[,] names = new string[2, 4];
```

```
int[,] matrix = new int[3, 3];
matrix[0, 1] = 5;
Console.WriteLine(matrix[0, 1]);
```

Най-важното правило е, че елементите **започват от 0**, ред -> колона

```
int[,] matrix = {
    {9, 2, 3}, //започват от [0,0] = първи ел.
    {4, 5, 6},
    {7, 8, 1}
};

string[,] names = new string[2, 4]; // задава размер

int element = matrix[1, 2]; // 6 (ред 1, колона 2)

Console.WriteLine(matrix[0, 0]); // първи ел 9

matrix[2, 1] = 100; // Променя 8 на 100

for (int row = 0; row < matrix.GetLength(0); row++) // обхождане
{
    for (int col = 0; col < matrix.GetLength(1); col++)
    {
        Console.Write(matrix[row, col] + " ");
    }
    Console.WriteLine();
}
```

```
int[,] matrix = {
    {9, 2, 3}, //започват от [0,0] = първи ел.
    {4, 5, 6},
    {7, 8, 1}
};
```

```

string[,] names = new string[2, 4]; // задава размер
int element = matrix[1, 2]; // 6 (ред 1, колона 2)
Console.WriteLine(matrix[0, 0]); // първи ел 9
matrix[2, 1] = 100; // Променя 8 на 100

for (int row = 0; row < matrix.GetLength(0); row++) // обхождане
{
    for (int col = 0; col < matrix.GetLength(1); col++)
    {
        Console.Write(matrix[row, col] + " ");
    }
    Console.WriteLine();
}

```

Друго, с което може да допълним е, че можем да създаем разбъркана редица или разбъркаме вече съществуваща с `Random random = new Random();` който работи с вграден метод `.Next()` с няколко overloads. Ако му зададем параметри `.Next(1,10)` трябва да знаем, че числото 1 ще се включи в обработката на интервала, а вместо 10, ще бъде числото преди него 9 за край, т.е. `[a, b)`

```
Random random = new Random();
```

нов разбъркан обект

```
var shuffled = numbers4v.OrderBy(x => random.Next()).ToArray();
```

```
Console.WriteLine("Разбъркан с LINQ: "
```

▲ 3 of 3 ▼ `int Random.Next(int minValue, int maxValue)`
 Returns a random integer that is within a specified range.
minValue: The inclusive lower bound of the random number returned.

```

Random random = new Random();

var shuffled = numbers4v.OrderBy(x => random.Next()).ToArray();

Console.WriteLine("Разбъркан с LINQ: " + string.Join(", ", shuffled));

```

Който има желание, да си го проучи сам.

Последно, понеже нашият код стана много дълъг, можем да си стартираме отделни парчета от него с #if

```
#define DEBUG или START, PART1, IGNORE са подходящи
using System.Numerics;

namespace Lists
{
    0 references
    internal class Program
    {
        0 references
        static void Main()
        {
            Кодът преди #if се изпълнява винаги - аз съм оставила заглавна лента
            Console.WriteLine("##### МАСИВИ - преговор с допълнение #####"); //винаги
            #if IGNORE този код е верен, искам да се игнорира, затова му слагам различно име от #define
            ... Ако напиша #if DEBUG ще стартира него
            #else
            ... ако предходната част е игнорирана, стартира else
            #endif
        }
    }
}
```

Изпълнява се заглавната лента – тя е оцветена обичайно, а игнорирания код става сив и няма IntelliSense.

```
1  #define DEBUG
2  using System.Numerics;
3
4  namespace Lists
5  {
6      0 references
7      internal class Program
8      {
9          0 references
10         static void Main()
11         {
12             Console.WriteLine("##### МАСИВИ - преговор с допълнение #####"); //винаги
13             #if IGNORE
14             СИВО // обикновено имената са 8 мн.ч. numbers, fruits, cars...
15             - няма int[] numbers1v = new int[5]; // нов масив вариант 1: 1v
16             да се int[] numbers2v; // деклариране
17             изпълни numbers2v = new int[6]; // 6 елемента, по подразбиране са 0-и
18         }
19     }
20 }
```

Мога да го свия да не пречи. Сега програмата ще прескочи IGNORE и ще започне от #else

```
#define DEBUG
using System.Numerics;

namespace Lists
{
    0 references
    internal class Program
    {
        0 references
        static void Main()
        {
            Console.WriteLine("##### МАСИВИ - преговор с допълнение #####"); //винаги

            #if IGNORE
            ...
            #else
            Стартира
            от тук -
            обичайни
            цветове
            Console.WriteLine("Въвегеме редица: ");
            int[] numbers4v = Console.ReadLine().Split().Select(int.Parse).ToArray();
            Console.WriteLine(numbers4v); // System.Int32[]
            Console.WriteLine(string.Join(' ', numbers4v));
            Array.ForEach(numbers4v, x => Console.Write(x));
        }
    }
}
```

Ако искам да работи кодът, който е игнориран, а другият не – правя така, че името в #if да съвпадне с #define – в случая DEBUG. Цветът е активен.

```
#define DEBUG
using System.Numerics;

namespace Lists
{
    0 references
    internal class Program
    {
        0 references
        static void Main()
        {
            Console.WriteLine("##### МАСИВИ - преговор с допълнение #####"); //винаги

            #if DEBUG // вече съвпадна с #define
            // обикновено имената са в мн.ч. numbers, fruits, cars...
            int[] numbers1v = new int[5]; // нов масив вариант 1: 1v

            int[] numbers2v; // деклариране
            numbers2v = new int[6]; // 6 елемента, по подразбиране са 0-и
        }
    }
}
```

Може да си поиграем и тестваме също със заключителна лента за край на урока, важното е дапомним, че кодът, който е извън #if-#endif участъка (заглавна и крайна лента) се изпълняват ВИНАГИ. Вътре в #-проверката кой участък ще се изпълни, зависи от това кое име сме дефинирали.

```

1  #define START
2  using System.Numerics;
3
4  namespace Lists
5  {
6      0 references
7      internal class Program
8      {
9          0 references
10         static void Main()
11         {
12             Console.WriteLine("##### МАСИВИ - презовор с допълнение #####"); //винаги
13             #if IGNORE
14             ...
15             #else
16             кодът извън #if-#endif се изпълнява ВИНАГИ,
17             ...
18             #endif
19             Console.WriteLine("##### КРАЙ - това беше!:) #####"); //винаги
20         }
21     }
22 }

```

Може също да закоментираме дефиницията и тогава ще се изпълнява директно #else.

```

1  // #define START
2  using System.Numerics;
3
4  namespace Lists
5  {
6      0 references
7      internal class Program
8      {
9          0 references
10         static void Main()
11         {
12             Console.WriteLine("##### МАСИВИ - презовор с допълнение #####"); //винаги
13             #if IGNORE
14             ...
15             #else
16             без #define отива в #else
17             ...
18             #endif
19             Console.WriteLine("##### КРАЙ - това беше!:) #####"); //винаги
20         }
21     }
22 }

```

#if директивите са чисто **compile-time** функция - те определят кой код да се компилира, преди програмата да започне изпълнение:

- **Compile Time:** "Мислене" на програмата - анализ, проверки, оптимизации
- **Runtime:** "Действие" на програмата - изпълнение, взаимодействие, изчисления

Аспект	Compile Time	Runtime
Кога се случва	Преди стартиране	По време на изпълнение
Грешки	Синтаксис, типове	Изключения, логически
#if директиви	✅ Обработват се	❌ Не се обработват
Променливи	Декларации и типове	Стойности и изчисления
Входни данни	Недостъпни	Достъпни (потребител, файлове)